

# Proctored AI Interview System — Design Document

*Full-fledged remote interview with live proctoring, environment lockdown, and JD/resume-driven question curation*

**Note:** This is a rebuilt draft from a verbal description, not a recovery of the original document. Specific choices (libraries, thresholds, exact flow) should be corrected against your real design.

---

## 1. Purpose

Conduct a complete remote interview while continuously verifying integrity: the candidate's camera and microphone are accessed from their PC, the exam environment is locked down, and the system monitors for anomalies (extra voices, background disturbance, poor lighting, looking away, second person present) in real time. Questions are curated per candidate from the job description and their resume, and the flow of answers is tracked throughout.

---

## 2. High-Level Architecture

| Layer                                        | Responsibility                                                                       |
|----------------------------------------------|--------------------------------------------------------------------------------------|
| <b>Candidate client</b><br>(browser/desktop) | Camera/mic capture, environment lockdown, in-browser detection, event streaming      |
| <b>Signal/streaming layer</b>                | Real-time transport of media + proctoring events (WebSocket / WebRTC)                |
| <b>Backend</b> (FastAPI)                     | Session lifecycle, question curation, answer capture, integrity scoring, audit trail |
| <b>Storage</b>                               | Media recordings, event logs, per-candidate evaluation                               |

---

## 3. Environment Lockdown

The goal is a controlled, kiosk-like exam window that makes casual cheating hard and flags tampering.

- **Camera & mic access** — captured from the candidate's PC via `getUserMedia` / WebRTC,

gated behind explicit consent.

- **Fullscreen enforcement** — interview runs in enforced fullscreen; exiting fullscreen raises a flag.
- **Focus / tab-switch detection** — window blur, tab switches, and visibility changes are logged as integrity events.
- **Input restrictions** — copy/paste, right-click context menu, and text selection disabled during the session.
- **Dev-tools / inspection detection** — attempts to open developer tools flagged.
- **Multiple-display detection** — detect external monitors where the platform allows it.
- **Session lock** — the interview "locks" the workspace so the candidate can't freely navigate away without generating events.

Browser-based lockdown has inherent limits; a dedicated desktop/kiosk client gives stronger guarantees if you need them.

---

## 4. Live Proctoring Signals

Detection ideally runs **in-browser** for low latency (e.g. MediaPipe / TensorFlow.js / face-api.js), with events streamed to the backend for scoring and audit.

### Visual

- **Face presence** — candidate must be visible; absence flagged.
- **Multiple-face detection** — a second person in frame is a high-severity flag.
- **Head pose / gaze** — sustained looking away from screen flagged.
- **Object detection** — phone or secondary device in frame flagged.
- **Lighting condition** — too dark, backlit, or overexposed frames detected and surfaced (candidate can be prompted to fix lighting).

#### Audio

- **Speech / voice activity** — detect when the candidate is speaking.
- **Background voice detection** — another speaker suggests coaching; flagged.
- **Disturbance / noise level** — sustained background noise or disruptions logged.

Each event carries a **type, timestamp, and severity**, so the final review isn't a black box.

---

## 5. Interview Flow & Question Curation

- **JD + resume ingestion** — the job description and candidate resume drive question selection, so each interview is tailored rather than generic.
- **Curated question set** — questions generated/selected against JD requirements and resume signals (skills, experience gaps, claimed projects).
- **Answer flow tracking** — answers captured in sequence, keyed to their question, with timing; the flow of responses is recorded for review and scoring.

*(This ties naturally to your existing resume-scoring rubric and JD handling on TalentAgent — the same JD/resume inputs can feed question curation here.)*

---

## 6. Integrity Scoring & Review

- **Real-time flags** aggregated over the session with severity weighting.
  - **Integrity summary** per candidate: what fired, when, how often.
  - **Human-review bucket** — borderline sessions routed to a reviewer rather than auto-decided.
  - **Audit trail** — append-only event log so any decision is traceable and defensible.
-

## 7. Suggested Tech Stack

| Component            | Option                                                  |
|----------------------|---------------------------------------------------------|
| Media capture        | WebRTC / <code>getUserMedia</code>                      |
| In-browser detection | MediaPipe, TensorFlow.js, or face-api.js                |
| Real-time transport  | WebSockets (events) + WebRTC (media)                    |
| Backend              | FastAPI (consistent with your current stack)            |
| Data contracts       | Pydantic                                                |
| Audio analysis       | VAD + background-speaker/noise detection                |
| Storage              | Media store + append-only event log + evaluation record |

## 8. Privacy & Consent

- **Explicit consent** before any camera/mic access, with clear disclosure of what's recorded and monitored.

- **PII handling** — reuse the Presidio-based masking approach for any candidate data leaving the interview boundary.
  - **Data retention policy** — define how long recordings and event logs are kept.
  - **Bias / fairness review** — proctoring signals can disadvantage candidates with poor equipment or noisy environments; keep a human in the loop and document mitigations.
- 

*Rebuilt from a description — reconcile with your original design when you're back at your machine.*